

CTF Write-up: Rust Reverse Engineering

Solver

April 2026

1 Informasi Challenge

- **Kategori:** Reverse Engineering
- **Bahasa Pemrograman:** Rust
- **Format Flag:** LKS2026{...}

2 Pendahuluan

Tantangan ini mengharuskan kita untuk melakukan *reverse engineering* pada sebuah program *binary* yang dikompilasi menggunakan bahasa pemrograman Rust. Tujuan utamanya adalah menemukan kunci enkripsi statis yang digunakan untuk menyembunyikan *flag* di dalam program. Bahasa Rust dikenal memiliki struktur *binary* yang cukup kompleks dan besar saat didekompilasi, namun logika inti dari *challenge* pemula biasanya terisolasi di dalam fungsi utama.

3 Metodologi Analisis

3.1 Identifikasi Awal

Langkah pertama dalam menganalisis program ini adalah memuat *binary* ke dalam *disassembler/decompiler* seperti Ghidra atau IDA Pro. Karena program dikompilasi dengan Rust, kita langsung mengabaikan fungsi-fungsi *wrapper* bawaan Rust (seperti `std::rt` atau `core::panicking`) dan mencari fungsi inti dari logika program.

Fungsi utama yang menjadi target kita terletak pada `main::main`.

3.2 Analisis Dekompilasi

Saat memeriksa *pseudocode* dari `main::main`, kita menemukan struktur logika yang cukup sederhana yang menunjukkan proses validasi kata sandi (*password checker*).

```
1 void __rustcall main::main(void)
2 {
3     byte local_e0 [32];
4     // ... [Deklarasi variabel lain] ...
5
6     // Inisialisasi array target
7     local_e0[0] = 0x6f;
8     local_e0[1] = 0x68;
9     local_e0[2] = 0x70;
10    local_e0[3] = 0x11;
11    local_e0[4] = 0x13;
```

```

12 // ... [Data array hingga indeks ke-31] ...
13
14 <>::from_str("Rust binary loaded\n", 0x13);
15 std::io::stdio::_print();
16 <>::from_str("key:\n", 5);
17 std::io::stdio::_print();
18
19 // ... [Proses membaca input dari user (stdin)] ...

```

Listing 1: Potongan fungsi main::main

Dari kode di atas, kita dapat mengamati bahwa program menyimpan sebuah *array byte* berukuran 32 elemen (`local_e0`) di dalam memori saat aplikasi dijalankan. Array ini kemungkinan besar merupakan bentuk terenkripsi dari *flag* yang kita cari.

3.3 Menganalisis Algoritma Verifikasi

Setelah membaca *input* dari pengguna, program melakukan perhitungan panjang *string*.

```

1 lVar1 = <>::len();
2 if (lVar1 == 0x20) { // 0x20 adalah 32 dalam desimal
3     // Lanjut ke verifikasi

```

Listing 2: Verifikasi Panjang Input

Program memastikan bahwa panjang input dari *user* harus persis 32 karakter (0x20 dalam heksadesimal). Ini mengonfirmasi bahwa panjang *flag* yang sebenarnya adalah 32 karakter, sesuai dengan ukuran *array* `local_e0`.

Selanjutnya, kita memeriksa blok perulangan (*loop*) yang membandingkan setiap karakter input dengan *array target*:

```

1 while (<>::next(&local_50,&local_68), (local_50 & 1) != 0) {
2     if (0x1f < local_48) {
3         core::panicking::panic_bounds_check();
4     }
5     if ((local_40 ^ 0x23) != local_e0[local_48]) {
6         local_99 = 0; // Flag salah
7     }
8 }

```

Listing 3: Loop Validasi Karakter

Di sinilah inti dari kelemahan kriptografi program ditemukan. Program mengambil setiap karakter (*byte*) dari *input user* (`local_40`), melakukan operasi logika **XOR (^)** dengan kunci statis 0x23, dan membandingkan hasilnya dengan *array* terenkripsi (`local_e0[local_48]`).

4 Eksploitasi dan Dekripsi

Berdasarkan sifat operasi XOR yang simetris ($A \oplus B = C \iff C \oplus B = A$), kita dapat membalikkan operasi verifikasi tersebut untuk mendapatkan *flag* asli.

Rumus dekripsi yang digunakan adalah:

$$Plaintext_Flag = Array_Terenkripsi \oplus Kunci_XOR$$

$$Plaintext_Flag = local_e0 \oplus 0x23$$

4.1 Skrip Solver (Python)

Kita mengekstrak seluruh nilai *hexadecimal* dari array `local_e0` dan menulis skrip Python sederhana untuk melakukan operasi XOR secara otomatis.

```
1 # Array data hex yang diekstrak dari fungsi main::main
2 data = [
3     0x6f, 0x68, 0x70, 0x11, 0x13, 0x11, 0x15, 0x58,
4     0x51, 0x56, 0x50, 0x57, 0x7c, 0x51, 0x46, 0x55,
5     0x46, 0x51, 0x50, 0x46, 0x7c, 0x46, 0x45, 0x11,
6     0x40, 0x12, 0x1a, 0x42, 0x41, 0x17, 0x14, 0x5e
7 ]
8
9 # Kunci statis yang ditemukan di dalam loop perbandingan
10 key = 0x23
11
12 flag = ""
13 for byte in data:
14     flag += chr(byte ^ key)
15
16 print(f"Flag yang ditemukan: {flag}")
```

Listing 4: Skrip Dekripsi Flag

Ketika skrip tersebut dijalankan, operasi *XOR Single-Byte* akan mengonversi setiap *byte hexadecimal* kembali menjadi karakter ASCII yang dapat dibaca manusia.

5 Kesimpulan

Tantangan ini menggunakan teknik enkripsi dasar berupa *Single-Byte XOR* untuk menyembunyikan *flag* di dalam *binary*. Meskipun program ditulis dengan bahasa Rust yang modern, logika dasar verifikasi tetap terekspos dalam proses dekompilasi. Dengan menganalisis fungsi `main::main`, menemukan *array ciphertext*, dan mengidentifikasi kunci XOR (0x23) pada blok perbandingan, *flag* dapat didekripsi dengan mudah menggunakan operasi aljabar sederhana.

Flag Akhir: LKS2026{rust_reverse_ef2c19ab47}